Can a computer recognise quick doodles?

[Name 1, SRN] · [Name 2, SRN]

- Problem: name the object in a quick hand drawing
- Why it matters: drawing could become a search interface
- Data: Google QuickDraw, 28×28 grayscale; 3 / 10 / 50 classes, 20,000 each
- Split per class: 16,000 train / 2,000 validation / 2,000 test



Our task: reproduce the paper's results, then add one new model.

Takeaway: same simple drawings, but harder as the number of classes grows.

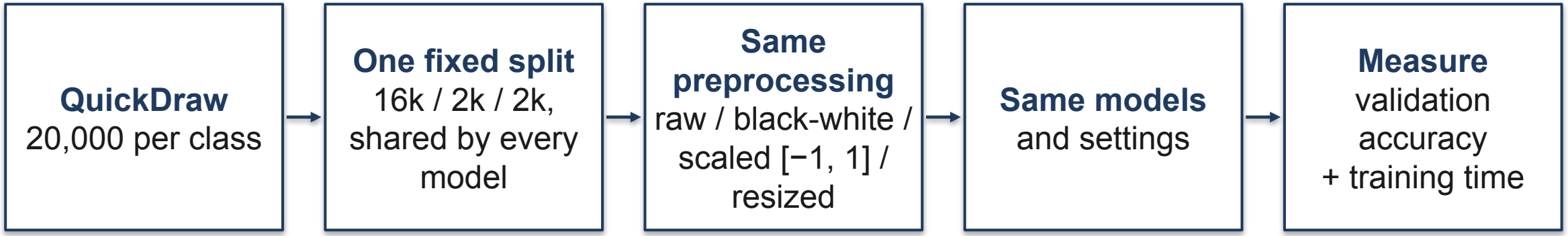
The original team tried four model families, from simple to complex

	Logistic Regression	SVM	CNN	Transfer learning
How it works	Draws straight-line boundaries between classes on raw pixels	Finds the widest margin between classes; kernels allow curved boundaries	Small filters slide over the image to detect strokes and shapes	Reuses networks pre-trained on 1.2 M photos (ImageNet); trains only a small new head
Why they used it	Fast baseline	Tests if non-linear boundaries help	Learns spatial patterns	Tests if photo knowledge helps doodles
Key settings	100 iterations	4 kernels, 500 iterations; 3 and 10 classes only	V1 = 2 conv + 2 pooling layers; V2–V4 = simpler versions; 100 epochs; raw and black/white input	VGG19, InceptionV3, ResNet50, MobileNet; 3 classes

Measured: validation accuracy and training time.

Takeaway: does a more complex model really help on simple drawings?

We rebuilt their exact pipeline



What differed

- Newer hardware (RTX 4070 laptop GPU) and newer library versions
- Transfer learning: frozen-network features computed once, identical result
- 30.9 min instead of about 15.7 h with their method
- We read the authors' original code to explain any gaps

Takeaway: same experiment, so any difference must have a cause.

Most results reproduce; three don't

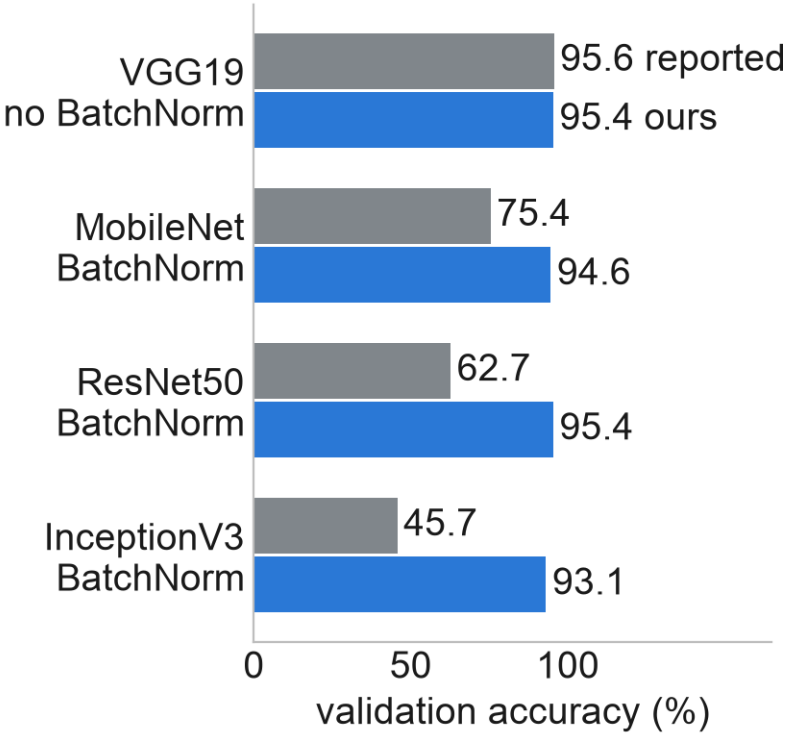
Model (classes)	Reported	Ours	Status
Logistic Regression (3 / 10 / 50)	79.5 / 64.6 / 43.9	79.3 / 65.8 / 43.3	**Matches: within 1.2 points**
CNN V1 (50)	81.8	81.2	**Matches**
VGG19 transfer learning (3)	95.6	95.4	**Matches**
SVM RBF (3)	62.7	84.0	**Differs: ours higher**
CNN "V2" (50)	81.8	75.1	**Differs: ours lower**
ResNet50 / InceptionV3 / MobileNet (3)	62.7 / 45.7 / 75.4	95.4 / 93.1 / 94.6	**Differs: ours much higher**

Validation accuracy (%), as in the paper.

Takeaway: the reproduction works; three gaps needed explaining.

Why the three differences happen

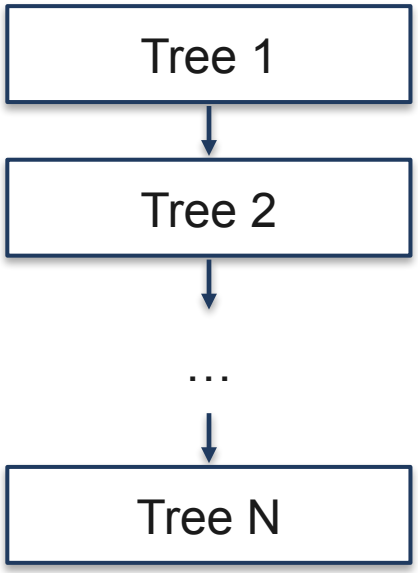
What we saw	Why	Our evidence
SVM scores higher than reported	Their runs used ~70,000 drawings per class with the same 500-step limit: less converged	Our 70,000 rerun: RBF 84.0 → 75.0 (3 classes), 74.6 → 64.3 (10); theirs 62.7 / 61.0
Their "V2" equals our V1	A script bug trained V1 under every V2–V4 label	Our V1 81.2 vs their "V2" 81.8; our real V2 75.1
Transfer learning far higher for 3 of 4 networks	Frozen BatchNorm layers behaved differently in 2018 Keras	VGG19 (no BatchNorm) matches: 95.4 vs 95.6; their Inception log: 99.99% train vs 45.7% validation



CNN V3 is 2.5 / 5.2 points lower at 10 / 50 classes: unexplained, likely seed or library differences.

Takeaway: the paper's claim that complex networks don't help doodles is likely an artefact.

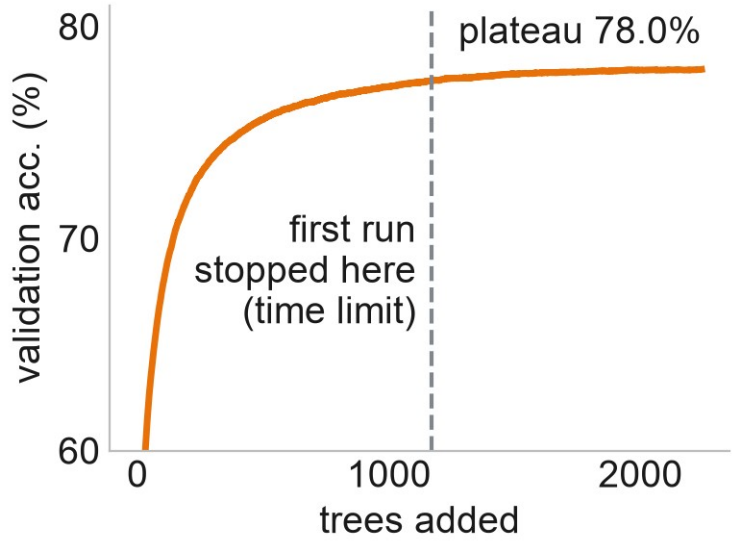
Our extension: XGBoost, a model family the paper never tried



Each new tree fixes the mistakes of the previous ones; their votes are added up.

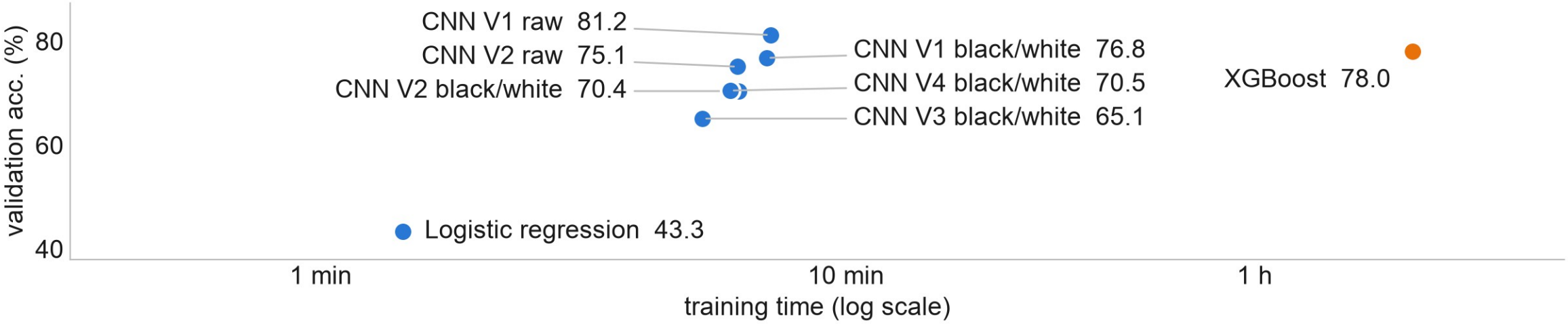
- Why: the reference had no tree-based model; it trains fast on a GPU
- Tuned 7 settings with Optuna Bayesian search, on validation only
- Test set used once; first run was time-limited, so we retrained fully

Classes	First run (time-limited) validation / test	Fully trained validation / test
3	91.7 / 90.8	91.7 / 90.8
10	89.8 / 89.7	89.8 / 89.8
50	77.4 / 77.4	78.0 / 77.9



Takeaway: more training gained only +0.5 points (test); XGBoost hits a ceiling around 78%.

All models compared: the CNN wins on 50 classes, accurately and fast



Classes	Logistic Regression	SVM	CNN	Transfer learning	XGBoost
3	79.3% · 20 s	84.0% · 41 s	91.7% · 38 s	95.4% · 3.5 min	91.7% · 57 s
10	65.8% · 20 s	74.6% · 10.9 min	88.8% · 1.8 min	—	89.8% · 8.1 min
50	43.3% · 86 s	—	81.2% · 7.2 min	—	78.0% · 119.9 min

Best model per family, validation %. — = not run in the reference (SVM: 3 and 10 classes; transfer learning: 3 classes).

Takeaway: simple models fall off as classes grow; CNNs scale best.

What we learned

- CNN is best on 50 classes: 81.2%, about 16.7× faster than XGBoost (78.0%)
- XGBoost matches CNNs on 3 and 10 classes; trees judge pixels, CNNs learn shapes
- Three surprising paper results traced to their setup: data size, script bug, BatchNorm
- Future work: feed transfer-learning features into XGBoost

Takeaway: yes: a small CNN names the right doodle out of 50 classes 81.2% of the time.